

LAB 3

Long Running Tasks and Data: ROS2 Actions, ROS Bags and QoS Inspection

Ricardo Severino

In the previous laboratories, you first observed the IASYS autonomous vehicle as a complete system and then opened the “black box” to understand how ROS 2 nodes, topics, services and parameters are used to connect its different components. In this laboratory, we will move one step further and look at two important ROS 2 mechanisms used when autonomous systems must execute and analyse longer-running tasks: **Actions** and **ROS Bags**.

You will extend the IASYS mini-stack with a navigation Action that allows a client to request a destination, receive continuous feedback while the vehicle is moving, obtain a final result when the mission is completed, and cancel the mission if necessary. This will help illustrate why Actions are better suited than Topics or Services for operations that take time to complete.

You will also record a complete autonomous mission using **ros2 bag**, inspect the recorded data and replay it afterwards. This will introduce an important capability for debugging, validation and offline analysis of autonomous systems.

Finally, you will briefly inspect the **QoS settings** used by different ROS 2 topics and relate them to the communication requirements of the system.

GOALS

- Understand when ROS 2 Actions are preferable to topics or services.
- Inspect and invoke a custom navigation Action in the IASYS mini autonomous system.
- Observe goal acceptance, periodic feedback, result delivery and cancellation.
- Relate the `rclcpp_action` C++ API to the action behaviour visible through the ROS 2 CLI.
- Complete a C++ Action client that sends navigation goals and processes feedback/results.
- Record an autonomous mission with `rosbag2`, inspect the recording and replay it in `RViz2`.
- Inspect QoS policies and relate them to different kinds of autonomous-system information.

1 FROM LAB2 TO LAB3

LAB2 opened the ROS 2 black box: you inspected nodes and topics, used a service and parameters, and created a C++ node. LAB3 keeps the same autonomous-system demonstrator but adds two mechanisms needed in larger systems: long-running task interfaces and data recording/replay.

Interface	Communication pattern	Typical IASYS example	Cancellation
Topic	Publish/subscribe, continuous stream	/odom, /cmd_vel, /path	No
Service	Request/response, short operation	/reset_vehicle	No
Action	Goal + feedback + result, long operation	navigate vehicle to a goal	Yes

A navigation mission is a natural Action: it takes time, the caller benefits from progress feedback, and the task may need to be canceled before completion.

2 ADD THE LAB3 PACKAGES TO THE WORKSPACE

Copy the supplied student starter packages into the IASYS workspace. Keep the LAB1/LAB2 iasys_demo package in place.

```
source /opt/ros/jazzy/setup.bash

cp -r <LAB3-resources>/lab3_student_starter/iasys_interfaces ~/iasys_ws/src/
cp -r <LAB3-resources>/lab3_student_starter/iasys_mission ~/iasys_ws/src/

cd ~/iasys_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install --packages-select iasys_interfaces iasys_mission
source install/setup.bash
```

The first package contains a custom ROS interface. The second contains the navigation Action server and a starter C++ Action client.

```
ros2 pkg list | grep iasys
ros2 interface show iasys_interfaces/action/NavigateToPose
```

3 UNDERSTAND THE CUSTOM ACTION

Open `action/NavigateToPose.action`. An Action definition contains three sections separated by ---:

```
float64 x
float64 y
---
bool success
float64 final_distance
string message
```

```
---
float64 distance_remaining
```

Section	Direction	Meaning in this LAB
Goal	client -> server	Target x/y position
Result	server -> client	Success/abort information and final position error
Feedback	server -> client, repeatedly	Current distance remaining to the target

Question: why would `/reset_vehicle` remain a service while `NavigateToPose` is better represented as an Action?

4 START THE SYSTEM AND THE ACTION SERVER

Use separate terminals. Source the Jazzy underlay and the IASYS overlay in every terminal.

```
# Terminal A - autonomous-system demonstrator
source /opt/ros/jazzy/setup.bash
source ~/iasys_ws/install/setup.bash
ros2 launch iasys_demo demo.launch.py

# Terminal B - mission Action server
source /opt/ros/jazzy/setup.bash
source ~/iasys_ws/install/setup.bash
ros2 run iasys_mission navigate_action_server
```

Inspect the Action without opening its implementation:

```
ros2 action list -t
ros2 action info /navigate_to_pose
ros2 node info /navigate_action_server
```

Record which topics the Action server also uses to connect the mission-level request to the existing planner/controller stack.

Observed element	Role
<code>/navigate_to_pose</code>	Mission-level Action interface
<code>/goal_pose</code>	Goal forwarded to the existing planner
<code>/odom</code>	Vehicle state used to compute feedback
<code>/path</code>	Observed for valid planning; an empty path is also used to stop on cancellation

5 SEND A GOAL AND OBSERVE FEEDBACK

```
ros2 action send_goal /navigate_to_pose iasys_interfaces/action/NavigateToPose
"{x: 6.0, y: -6.0}" --feedback
```

Observe all three phases:

1. the *goal* is accepted by the server;
2. *distance_remaining* is reported repeatedly while the vehicle moves;
3. a final result is returned when the vehicle reaches the requested position.

Try a second reachable goal. While the first goal is still executing, try to send another goal from a different terminal. The supplied server rejects concurrent missions rather than silently replacing the current mission. Discuss whether another system might instead choose preemption.

6 CANCELLATION: A KEY ACTION PROPERTY

Send a goal that takes several seconds to reach:

```
ros2 action send_goal /navigate_to_pose iasys_interfaces/action/NavigateToPose
"{x: 6.0, y: 6.0}" --feedback
```

While the goal is executing, press Ctrl+C in the `send_goal` terminal. In the current Jazzy `ros2action` tool, SIGINT requests cancellation of the active goal. The server accepts the cancel request and publishes an empty `/path` so the existing controller stops the vehicle.

Answer briefly:

- How is cancellation different from merely terminating the client process?
- Why should a long-running autonomous task define an explicit cancellation behaviour?
- What would be unsafe about canceling the Action while leaving the previous motion command active?

7 OPEN THE C++ ACTION SERVER

Now inspect `src/navigate_action_server.cpp`. Find the `create_server` call and the three server callbacks:

```
handleGoal(...)      -> accept or reject a requested goal
handleCancel(...)     -> accept or reject a cancellation request
handleAccepted(...)   -> start execution of an accepted goal
```

The actual mission execution runs in a separate thread. This is important: a long-running operation must not block the executor callback that accepted the goal.

During execution, the server publishes the target to `/goal_pose`, monitors `/odom`, publishes *feedback*, and eventually calls one of:

- `goal_handle->succeed(result);`
- `goal_handle->canceled(result);`
- `goal_handle->abort(result);`

Code-reading question: why does the Action server not directly implement A* or the motion controller? Relate your answer to modularity and stable interfaces.

8 COMPLETE THE C++ ACTION CLIENT

The supplied student version of *src/navigate_action_client.cpp* compiles but contains TODOs. Complete it so the executable receives X and Y from the command line, sends the goal, prints feedback, and reports the final result.

Your implementation must:

- create an rclcpp_action client for NavigateToPose on /navigate_to_pose;
- wait for the server for up to 5 seconds;
- populate NavigateToPose::Goal with the command-line coordinates;
- define goal-response, feedback and result callbacks;
- call `async_send_goal(...)` and spin until the result callback shuts ROS down.

Useful API fragments (not a complete solution):

```
auto client = rclcpp_action::create_client<NavigateToPose>(node,
"/navigate_to_pose");

auto options = rclcpp_action::Client<NavigateToPose>::SendGoalOptions();
options.goal_response_callback = ...;
options.feedback_callback = ...;
options.result_callback = ...;

client->async_send_goal(goal, options);
```

Rebuild only the relevant packages and test your client:

```
cd ~/iasys_ws
colcon build --symlink-install --packages-select iasys_interfaces iasys_mission
source install/setup.bash

ros2 run iasys_mission navigate_action_client 5.5 -5.5
```

Use `ros2 node info` while the client is running and compare the graph-level view with the C++ code.

9 RECORD A MISSION WITH ROSBAG2

ROS bags record selected ROS data with timestamps so that an experiment can be analysed or replayed later. This is especially useful for perception, localization, debugging and regression testing.

```
mkdir -p ~/iasys_bags
cd ~/iasys_bags

ros2 bag record -o mission_run \
  /odom /path /cmd_vel /goal_pose /trajectory \
  /vehicle_markers /planner_markers /world_markers /map
```

While recording, execute one complete navigation mission. Stop the recorder with `Ctrl+C` after the vehicle reaches the goal.

```
ros2 bag info ~/iasys_bags/mission_run
```

From the bag information identify: duration, storage backend, topics/types, and message count per topic. Jazzy normally uses MCAP as the default rosbag2 storage backend; do not expect the old ROS2 Foxy .db3 layout.

10 REPLAY THE MISSION WITHOUT RUNNING THE SYSTEM

Stop the demo and the Action server. The purpose now is to replay recorded information, not to execute the autonomous algorithms again.

```
# Terminal A - RViz only
source /opt/ros/jazzy/setup.bash
source ~/iasys_ws/install/setup.bash
rviz2 -d "$(ros2 pkg prefix iasys_demo)/share/iasys_demo/rviz/iasys_demo.rviz"

# Terminal B - replay
source /opt/ros/jazzy/setup.bash
source ~/iasys_ws/install/setup.bash
ros2 bag play ~/iasys_bags/mission_run
```

Repeat at half speed:

```
ros2 bag play ~/iasys_bags/mission_run --rate 0.5
```

Question: during replay, is A* planning being recomputed? Is the controller deciding new commands? Explain the difference between replaying recorded ROS messages and re-running the autonomous system.

11 INSPECT QoS CHOICES

DDS/ROS 2 QoS policies influence how data are delivered. Inspect two intentionally different types of information in the demo:

```
ros2 topic info /map -v
ros2 topic info /odom -v
```

Topic	Expected reliability	Expected durability	Why this makes sense
/map	RELIABLE	TRANSIENT_LOCAL	Map/state changes rarely; late joiners may need the latest retained sample
/odom	RELIABLE	VOLATILE	Live state stream; new subscribers normally care about current/future samples

Do not memorize these profiles as universal rules. The correct QoS depends on the application, network and criticality requirements. Later cooperation experiments will make these trade-offs observable under delay/loss.

Run a general ROS 2 environment report and identify the ROS distribution and middleware implementation used in your VM:

```
ros2 doctor --report
```

12 OPTIONAL EXPERIMENT: COMPARE TWO RUNS

If time remains, record two missions to the same destination using different controller speeds:

```
ros2 param set /simple_controller linear_speed 0.4
# record a mission as mission_slow

ros2 param set /simple_controller linear_speed 0.9
# record the same mission as mission_fast
```

Compare ros2 bag info for both recordings. Which measurements change? Which topics retain approximately the same geometry but different timing? This is a first example of using bags as reproducible experimental evidence.

13 LAB3 OUTCOMES AND NEXT STEPS

By the end of the LAB you should be able to provide:

- A Topics vs Services vs Actions comparison using examples from the IASYS mini-stack.
- Evidence of a successful */navigate_to_pose goal* showing *feedback* and a final result.
- Evidence that an executing navigation Action can be canceled and that the vehicle stops.
- A compiling C++ *navigate_action_client* that accepts X/Y command-line coordinates.
- One mission_run ROS bag and the corresponding ros2 bag info output.
- Short answers to the server modularity question and the bag replay question.
- QoS comparison for */map* and */odom*.

You now have the ROS 2 mechanisms needed to build the rest of the course: modular C++ nodes, topics, services, parameters, actions and reproducible recordings. The following laboratories will increasingly replace the simple black-box components with perception, localization, planning and control implementations.

14 REFERENCES AND SUPPORT LINKS

ROS 2 Jazzy documentation - Actions:
<https://docs.ros.org/en/jazzy/How-To-Guides/Topics-Services-Actions.html>

ROS 2 Jazzy tutorials: <https://docs.ros.org/en/jazzy/Tutorials.html>

ROS 2 rosbag2 documentation: <https://github.com/ros2/rosbag2>

IASYS Week 2 and 3 - ROS 2 lecture material.

Previous IASYS LAB3 material, adapted from Turtlesim/Fibonacci to the IASYS mini autonomous-system context.